ChatGPT

# VS Code Python Debugger & Python Virtual Environments: Executive Summary

Debugging and environment management are essential skills for any Python developer. A **debugger** is a tool that lets you run a program step-by-step and inspect its state (variables, call stack, etc.) at each point【30†L29-L33】. VS Code's Python debugger uses Microsoft's **debugpy** and the Python extension to provide a rich debugging experience in the editor. This lesson teaches you how to set up VS Code for Python, create and activate Python virtual environments (`venv`), and use the VS Code debugger features to find and fix bugs.

Key takeaways include:

- **Setting up VS Code for Python:** Install the Python extension, select a Python interpreter (global or virtual) via the status bar or Command Palette【6†L318-L326】, and understand workspace vs. global environments.
- **Creating virtual environments:** Use `python3 -m venv .venv` (macOS/Linux) or `py -m venv .venv` (Windows) to create an isolated project environment【11†L209-L218】. Activate with `source .venv/bin/activate` (Unix/macOS) or `.\.venv\Scripts\activate` (Windows)【11†L229-L238】. This keeps project dependencies separate.
- **Dependency management:** Install packages inside the activated venv with `pip`, and record them in **requirements.txt** (`pip freeze > requirements.txt`). Reinstall later with `pip install -r requirements.txt`【16†L15-L24】.
- **Launching the debugger:** Use the **Run and Debug** view or press **F5**【15†L190-L197】. If prompted, generate a `launch.json` (Python File)【1†L231-L239】. Choose "Integrated Terminal" or "External Terminal" in `launch.json` for program input【22†L617-L624】.
- **Breakpoints and stepping:** Set breakpoints by clicking in the gutter or pressing F9【15†L180-L187】. Control execution with **Continue (F5)**, **Step Over (F10)**, **Step Into (F11)**, **Step Out (Shift+F11)**【15†L232-L240】, and **Restart (Shift+F5)**.
- **Inspecting state:** Use the Debug sidebar to view **Variables**, **Call Stack**, and **Watch** panels【20†L408-L417】. The Debug Console lets you evaluate expressions or provide input for `input()` calls.
- **Advanced topics:** Conditional and function breakpoints【20†L362-L370】, data breakpoints, and logpoints help focus on complex bugs. Remote debugging is also possible via `debugpy` and SSH【22†L473-L476】.
- **Best practices:** Always use a venv per project【8†L51-L59】. Keep `venv` folders out of source control (add to `.gitignore`). Pin dependencies (`pip freeze`) for reproducibility. Use meaningful breakpoints and step through code instead of scattershot `print()` statements【30†L29-L33】【4†L182-L187】.

This lesson will guide you through all setup steps, provide annotated code examples to debug (functions, `input()`, list mutation, exceptions), explain `launch.json`, show OS-specific commands, and end with

troubleshooting tips and a short quiz. References are included throughout for official documentation and reliable guides.

---

# 1. Introduction to Debuggers and Virtual Environments

**Debugger definition:** A debugger is *"a tool that allows you to examine the state of a running program"*【30†L29-L33】. It lets you pause execution at **breakpoints**, inspect variable values, step through code line-by-line, and resume execution. This is crucial for finding bugs systematically, instead of relying on manual `print()` statements. A breakpoint acts as a marker telling the debugger *"pause when you reach this line"*【15†L180-L187】.

**Why use a debugger?** Debuggers save time and reduce errors by showing you exactly what your code is doing at runtime. You can view call stacks, inspect variables, and even modify values on the fly【20†L408-L417】. For example, if a loop or function isn't working correctly, you can set a breakpoint inside it and run the debugger to watch how data changes. This is much more efficient than interspersing lots of `print` calls or guesswork【30†L29-L33】.

**Virtual environments:** In Python, a *virtual environment (venv)* is a self-contained directory that contains its own Python interpreter and libraries. The standard library's `venv` module creates lightweight virtual environments on top of an existing Python installation【8†L51-L59】. Each venv has an isolated set of packages. By default, it is *"isolated from the packages in the base environment"* so it only uses packages you explicitly install into it【8†L51-L59】. This prevents version conflicts between projects. Virtual environments are typically stored in a project folder (often named `venv/` or `.venv/`) and should **not** be checked into version control【8†L69-L78】【11†L213-L221】. They are meant to be disposable: delete and recreate them as needed.

**Use-case:** Combining VS Code and venvs, you can maintain multiple projects with different dependencies. For each project, create and activate a venv. Configure VS Code to use that venv's interpreter. Then you can run and debug your code in a clean, controlled environment, ensuring consistency across development and deployment【8†L51-L59】【6†L323-L330】.

---

# 2. Setting Up VS Code for Python

## 2.1 Install VS Code and Extensions

1. **Download VS Code:** Install Visual Studio Code from code.visualstudio.com.
2. **Install Python:** Ensure Python is installed on your system (from python.org or via your OS package manager). Confirm in a terminal:

```
python --version   # On Windows, try `py --version`
```

1. **Install Python extension:** Open VS Code, click the **Extensions** icon (⇧⌘X / Ctrl+Shift+X), search for **Python** (by Microsoft), and install it. This provides IntelliSense, linting, and debugging support. VS Code automatically includes the **Python Debugger** extension (debugpy) with this【1†L198-L206】.

2. **Verify installation:** In VS Code's Extensions view, search for `@installed python`. You should see the **Python** and **Python Debugger** extensions listed【1†L198-L206】. If needed, restart VS Code.

### 2.2 Configure Python Interpreter

VS Code needs to know which Python interpreter to use for running and debugging your code. This can be a global Python install or a virtual environment.

- **Status Bar:** After opening a Python file, look at the bottom-left status bar in VS Code. It shows the active Python interpreter (e.g. `Python 3.x.x ('venv':venv)` or `Python 3.x (C:\Python3\python.exe)`).
- **Select Interpreter:** Click this interpreter version in the status bar (or press `Ctrl+Shift+P` and run **Python: Select Interpreter**). A list of discovered environments appears (Conda envs, venvs in workspace, system Python, etc.)【6†L323-L330】. Choose the one for your project. The chosen interpreter will be used for running code, debugging, and IntelliSense【6†L323-L330】.

  **Tip:** By default, if you open a workspace without selecting an interpreter, VS Code tries to auto-select in this order: (1) a workspace-local virtual environment (`.venv` or `venv` folder) and (2) a global/system interpreter【6†L338-L347】. You can override this by manually selecting an interpreter, or configuring the `python-envs.defaultEnvManager` setting.

- **Open Folder:** Always open your project as a VS Code *workspace folder* (via **File > Open Folder**). This ensures launch configurations (`launch.json`) are saved inside a `.vscode/` subfolder and that interpreter selection is per-workspace.

### 2.3 Example: Setting Interpreter

For example, you might create a venv named `.venv` in your project folder (see Section 3). After activating it (in a terminal), VS Code will detect it. Then select it via the status bar:

```
Enter the **Command Palette** (Ctrl+Shift+P), type “Python: Select Interpreter”, and pick “Python 3.x.x ('.venv':venv)”.【6†L323-L330】
```

This makes sure all run/debug actions use the Python inside `.venv`.

---

# 3. Creating and Managing Python Virtual Environments

A virtual environment (venv) isolates your project's Python interpreter and libraries from others on the system【8†L51-L59】. We will cover creating venvs on different OSes, activating/deactivating them, and managing dependencies.

## 3.1 Choosing a Tool

- **venv (built-in):** Since Python 3.3, the standard `venv` module is the recommended way to create virtual environments【8†L98-L106】. It has no external dependencies.
- **virtualenv (PyPI):** An older tool; it is no longer needed on Python≥3.3, since `venv` is built-in 【8†L111-L119】.
- **Conda (Anaconda):** If you use Anaconda/Miniconda, `conda create -n myenv python=3.x` can make envs. We focus on `venv` here, but note that Conda environments serve a similar purpose with their own commands.

Generally prefer `venv` for its simplicity and ubiquity.

## 3.2 Creating a Virtual Environment

Run one of the following commands in a terminal (in your project's root folder):

| OS | Command (in project folder) | Example Output |
|---|---|---|
| **Windows (cmd.exe)** | `py -m venv .venv` <br>(or `python -m venv .venv`) | Creates folder `.venv\`, with Scripts and Lib. |
| **Windows (PowerShell)** | `python -m venv .venv` <br>(if `py` not set up) | Creates folder `.venv\` similarly. |
| **macOS/Linux (bash/zsh)** | `python3 -m venv .venv` <br>(or `python -m venv .venv`) | Creates folder `.venv/`, with bin/ and lib/. |

- The argument `.venv` is the target directory. You can name it `venv` or anything, but `.venv` is common to mark it as hidden.
- This command sets up the new environment by copying (or symlinking) the Python interpreter into `.venv/bin/` (Unix) or `.venv\Scripts\` (Windows), and creating a `pyvenv.cfg` file【8†L98-L106】.
- If the folder already exists, you can use `--clear` or `--upgrade` flags per `venv` docs【8†L98-L106】.

**Example (macOS/Linux):**

```
$ cd my_project
$ python3 -m venv .venv
$ ls -d .venv/        # shows the new .venv directory
```

**Example (Windows):**

```
C:\> cd my_project
C:\my_project> py -m venv .venv
C:\my_project> dir .venv
 'Scripts'    'Include'    'Lib'    'pyvenv.cfg'
```

### 3.3 Activating and Deactivating

After creating a venv, you must **activate** it in each new terminal session to use its Python and `pip`. Activation prepends the venv's bin/Scripts directory to your PATH. Use the following commands:

| OS/Command Window | Activate Command | Deactivate Command |
|---|---|---|
| **Windows (cmd.exe)** | `.venv\Scripts\activate.bat` | `deactivate` |
| **Windows (PowerShell)** | `.\.venv\Scripts\Activate.ps1` | `deactivate` |
| **macOS/Linux (bash/zsh)** | `source .venv/bin/activate` | `deactivate` |

- On successful activation, your prompt will change (often prefixing with `(.venv)`).
- Check which Python is active: `which python` (Unix/macOS) or `where python` (Windows) should show the path inside `.venv` 【11†L242-L252】.
- Deactivate by running `deactivate` (or simply closing the shell). After deactivation, the original Python is restored.

After activation, installing packages via `pip install` will place them in the venv, not globally. This keeps project dependencies contained 【11†L262-L270】.

### 3.4 Pip and Dependency Management

Inside an active venv:

- **Upgrade pip:** (optional) `python -m pip install --upgrade pip` to ensure latest version.
- **Install packages:** Use `pip install package_name`. E.g. `pip install requests`.
- **Freeze requirements:** After installing needed libraries, export them with `pip freeze > requirements.txt`. This captures exact versions.
- **Install from requirements:** On another machine, or to reinstall, run:
- macOS/Linux: `python3 -m pip install -r requirements.txt`
- Windows: `py -m pip install -r requirements.txt` 【16†L15-L24】.

This reads the list of packages and versions and installs them into the active venv.

*Example*: If `requirements.txt` contains:

```
requests==2.18.4
numpy>=1.21
```

then `pip install -r requirements.txt` will install exactly those versions into your environment【16†L15-L24】.

- **Requirements vs pyproject.toml:** Traditionally, Python projects use `requirements.txt`. Newer projects may use `pyproject.toml` for build configuration (PEP 517/518), but most simple tutorials still rely on `requirements.txt`. For learning basics, focus on `requirements.txt`.

### 3.5 Workspace vs Global Environments

- A **workspace** environment is a venv located in your project folder. VS Code (by default) will prefer this for that project【6†L338-L347】.
- A **global/system** environment is a Python installation system-wide. You might use this for quick scripts, but it's not isolated.

  **Best Practice:** Use a separate venv for each project【8†L51-L59】. Do not install project dependencies globally. This avoids conflicts between projects.

- **Notes:**
- Don't commit the `.venv/` folder to Git. Instead, add it to `.gitignore` (or configure according to your VCS)【11†L209-L218】.
- If a venv becomes broken (e.g. missing `pyvenv.cfg` or corrupted), simply delete and recreate it with the above steps【6†L352-L360】.

---

## 4. Launching and Configuring the Debugger in VS Code

Now that Python and the environment are set up, we can configure VS Code's debugger.

### 4.1 The Run and Debug View

1. **Open the Debug view:** Click the *Run and Debug* icon (play button with bug) in the Activity Bar on the left side of VS Code, or press `Ctrl+Shift+D`.
2. **Start debugging:** If no launch configuration exists, you'll see a **Run and Debug** button. Clicking it (or pressing **F5**) prompts VS Code to select a debugger. Choose **Python File**. This automatically creates a `launch.json` (in `.vscode/`) with a basic configuration for the currently open Python file【1†L231-L239】.

Alternatively, use **Run > Open Configurations** to manually create or edit `launch.json` 【1†L231-L239】. The steps (from the official doc) are:

- Select the link or command to create a `launch.json`.
- Choose **Python Debugger** as the environment.

- Select **Python File** (or another suitable configuration, e.g. Django if working with a web app) 【1†L231-L239】.

- **Examine** `launch.json`**:** VS Code will open `.vscode/launch.json` with a template configuration. For example:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File (Integrated Terminal)",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}
```

- **type:** Should be `debugpy` for Python.
- **program:** `${file}` means the active file in the editor.
- **console:** `"integratedTerminal"` uses VS Code's terminal. You can also use `"externalTerminal"` (opens a new terminal window)【22†L617-L624】. If your program requires input via `input()`, using an integrated or external terminal is necessary (the Debug Console alone does not accept standard input by default)【22†L617-L624】【22†L534-L542】.
- You can add **args**, **cwd**, and other options as needed (covered later).

If you do not create a `launch.json`, pressing F5 will still start debugging the active file (with default settings) but having `launch.json` lets you customize options.

### 4.2 Running the Debugger

- **Start Debug Session:** With a Python file open and an interpreter selected, press **F5** or click the green **Start Debugging** button. VS Code may ask which debugger to use – pick **Python**.
- **The Debug Toolbar:** Once started, a floating debug toolbar appears with actions for **Continue**, **Step Over**, **Step Into**, **Step Out**, **Restart**, and **Stop**【15†L232-L240】.

The default debug console opens, the status bar turns orange, and the Debug sidebar updates. Breakpoints and variables will be active as the program runs.

### 4.3 Example `launch.json` Configurations

You can use and modify `launch.json` to customize debugging. Here are common examples:

- **Integrated vs External Console:**
  By default, the Python extension provides two pre-made configs (Integrated and External terminal)【22†L617-L624】:

```
{
  "name": "Python: Current File (Integrated Terminal)",
  "type": "debugpy",
  "request": "launch",
  "program": "${file}",
  "console": "integratedTerminal"
},
{
  "name": "Python: Current File (External Terminal)",
  "type": "debugpy",
  "request": "launch",
  "program": "${file}",
  "console": "externalTerminal"
}
```

Select the appropriate one in the debug dropdown (top of Run view) to switch. Use **integratedTerminal** to keep everything in VS Code; use **externalTerminal** if you prefer a separate OS terminal (useful if needing elevated privileges or to avoid issues with integrated console input)【22†L617-L624】【22†L584-L593】.

- **Passing Arguments:**
  You can pass command-line arguments via the `"args": []` property.

```
{
  "name": "Python: Current File with Args",
  "type": "debugpy",
  "request": "launch",
  "program": "${file}",
  "console": "integratedTerminal",
  "args": ["--mode", "test", "input.txt"]
}
```

Then `sys.argv` in Python will include these values.

- **Remote Attach (advanced):**
  For completeness, attaching to a remote Python process can be done with:

```
{
  "name": "Python: Attach to Remote",
  "type": "debugpy",
  "request": "attach",
  "port": 5678,
  "host": "localhost",
  "pathMappings": [
      {"localRoot": "${workspaceFolder}", "remoteRoot": "."}
  ]
}
```

Combined with running `debugpy.listen()` on the remote machine【22†L517-L524】. (See Section 8 for remote debugging.)

- **Why launch.json?**
  The `launch.json` file defines how VS Code runs and debugs your code【1†L216-L224】【22†L612-L621】. You can have multiple configurations in one file to debug different scripts or use different settings. The **DEBUG CONSOLE** status (in status bar) shows which config is active, and you can switch without reopening the Run view【15†L213-L218】.

---

## 5. Using the Debugger: Breakpoints, Stepping, and Inspection

Once the debugger is running, you control execution and inspect state using the VS Code debug UI.

### 5.1 Setting Breakpoints

- **Line Breakpoints:** Click in the editor gutter next to a line number, or press **F9** on a line to toggle a breakpoint【15†L182-L187】【15†L270-L278】. A red dot will appear. Execution will pause *before* that line is executed.
- **Conditional Breakpoints:** Right-click a breakpoint and select "Add Conditional Breakpoint." You can set a boolean expression or hit count. The debugger will only pause there if the condition is true【15†L182-L187】【20†L303-L312】.
- **Function Breakpoints:** In the BREAKPOINTS section of the Run view, click the `+` and choose **Function Breakpoint**, then enter a function name. Execution will break when that function is called, even if source is not loaded【20†L362-L370】.
- **Inline Breakpoints:** (For multi-statement lines, see [20])
- **Logpoints:** Right-click gutter, "Add Logpoint" to log a message (with optional expressions) instead of breaking【20†L380-L389】. This is like a dynamic `print`.

**Visual:** Breakpoints appear in the Run view's BREAKPOINTS list, where you can enable/disable or delete them. Hovering over a breakpoint in the editor shows its condition or log message.

### 5.2 Stepping Through Code

Use the debug toolbar or keyboard shortcuts to move through code:

| Action | Windows/Linux Shortcut | macOS Shortcut | Description |
|---|---|---|---|
| **Continue / Pause** | F5 | F5 | Resume execution until next breakpoint or pause debugging if running【15†L232-L240】. |
| **Step Over** | F10 | F10 | Execute the next line of code, but if it's a function call, run the entire function without diving in 【15†L232-L240】. |
| **Step Into** | F11 | F11 | Execute the next line, stepping into any function calls to see inside them【15†L232-L240】. |
| **Step Out** | Shift+F11 | ⇧F11 *(Mac same)* | Finish the current function and pause after returning to the caller【15†L232-L240】. |
| **Restart** | Shift+F5 | ⇧⌘F5 | Stop and start debugging again from the beginning (uses current config)【15†L246-L250】. |
| **Stop** | Shift+F5 | ⇧F5 | Terminate the debug session【15†L246-L250】. |
| **Toggle Breakpoint** | F9 | F9 | Add or remove a breakpoint on the current line. |

> **Note:** On some keyboards, you may need to press `Fn`+F5, etc. Also, VS Code shows these shortcuts in tooltips on the debug toolbar icons (and you can customize them).

The *Debug toolbar* (floating near the top center) has buttons for these actions. Hovering over each icon shows a tooltip with its name and shortcut【15†L232-L240】.

### 5.3 Inspecting Variables and Call Stack

During a paused session (when execution is stopped at a breakpoint), you can examine program state:

- **Variables Pane:** In the Run and Debug sidebar, the **VARIABLES** section lists local and global variables in the current scope【20†L408-L417】. It shows each variable name and current value. Expand complex objects to inspect fields. You can also hover your mouse over variables in the editor to see their values in a tooltip.

You can *edit* a variable's value on the fly: right-click a variable in VARIABLES and choose **Set Value**, or click the pencil icon to modify it【20†L408-L417】. This can be useful to test a fix without restarting.

- **Watch Pane:** In the **WATCH** section of the sidebar, you can enter expressions to monitor. Click the `+` and type a variable or expression. The debugger will evaluate these each step. This is handy for expressions that change often (e.g. `len(mylist)`).

- **Call Stack:** The **CALL STACK** section shows the stack of function calls leading to the current line 【20†L408-L417】. You'll see entries like `main()`, `some_function()`, etc. Click a stack frame to view local variables at that level. This lets you inspect variables in parent or child functions.

- **Debug Console:** The Debug Console (bottom panel) allows you to type and evaluate Python expressions at the current pause point【20†L433-L442】. For example, if paused in a function, typing `variable_name` and Enter will show its value. You can also modify variables by assignment (e.g. `x = 10`). When your program calls `input()`, VS Code will prompt in the Debug Console or terminal you chose. (If using `console: "integratedTerminal"`, input appears in the integrated terminal.)

Example interaction:

```
> Debug Console:
>> name = "Alice"    # examine or set variables
>> print(name)
Alice
```

- **Output:** The **DEBUG CONSOLE** also shows program output (stdout/stderr). Any `print` statements appear here. If you used an external terminal, output shows in that window instead.

### 5.4 Example: Step-by-Step

Consider this simple code in `calc.py`:

```python
def add(a, b):
    result = a + b   # (1)
    return result

x = 5              # (2)
y = 10             # (3)
sum_xy = add(x, y) # (4)
print("Sum is", sum_xy)  # (5)
```

**Debugging steps:**
1. Place a breakpoint at line (4) (`sum_xy = add(x, y)`) by clicking the gutter.
2. Start debugging (F5). Execution runs lines 1-3 normally, then pauses at line 4 before calling `add`.
3. Inspect Variables: The VARIABLES pane shows `x = 5`, `y = 10`, and (so far) no `sum_xy`.
4. Press **Step Into (F11)**: This enters the `add` function and highlights `result = a + b`. Now `a = 5`, `b = 10` are shown in VARIABLES.
5. Press **Step Over (F10)**: Executes `result = a + b` and moves to `return result`. Variables now show `result = 15`.
6. Press **Step Over** again: returns from `add` to the caller (line 4). Now `sum_xy = 15` appears.
7. Continue (F5): The program runs to the end and prints output.

This workflow is much faster than adding temporary prints in the code. You see variable values live, step through functions, and verify logic.

---

## 6. Debugging Examples

Below are several example programs that illustrate common scenarios (functions, input, list mutation, exceptions) and how you would debug them in VS Code. Set breakpoints as indicated and use the debugger steps discussed above.

### 6.1 Example 1: Debugging a Function

```
# file: greet.py
def greet(name):
    greeting = f"Hello, {name}!"
    return greeting  # <-- set breakpoint here

user = "Alice"
message = greet(user)
print(message)
```

- Set a breakpoint on `return greeting`. - Run the debugger. When paused, inspect `name` (should be "Alice") and `greeting` (`"Hello, Alice!"`) before the `return`. - Step over to return and see `message = "Hello, Alice!"` appear. - This confirms the function behavior.

### 6.2 Example 2: Debugging `input()` Handling

```
# file: echo.py
text = input("Enter something: ")  # <-- breakpoint here
print("You entered:", text)
```

- Place a breakpoint on the `input()` line. - Use **Integrated Terminal** in `launch.json` so you can type into the program. - Start debugging. The Debug Console will prompt `Enter something:`. Type a string (e.g. `testing`). - After input, VS Code will pause at the same line (if you break on it). You can inspect the `text` variable. Then press Continue to see the final print. - If the console is not accepting input, try the **External Terminal** configuration【22†L617-L624】.

### 6.3 Example 3: List Mutation

```
# file: list_mutation.py
def append_four(numbers):
    numbers.append(4)
    # <-- breakpoint on next line
    return numbers

nums = [1, 2, 3]
```

```
result = append_four(nums)  # Breakpoint stops after calling function
print("Result list:", result)
```

- Break on the line `return numbers`. - Step into `append_four`: observe `numbers = [1,2,3]`, then step over to append. After appending, `numbers = [1,2,3,4]`. - Note: The original list `nums` is modified as well, since lists are mutable. The VARIABLES pane will show `nums = [1,2,3,4]` outside the function after it returns. - This illustrates how changes in a function affect external variables.

### 6.4 Example 4: Exception/Error

```
# file: divide.py
def divide(a, b):
    result = a / b   # Potential ZeroDivisionError here
    return result

x = 10
y = 0  # <-- set y to 0 to trigger error
quotient = divide(x, y)  # <-- breakpoint before call
print("Quotient is", quotient)
```

- Set a breakpoint at the call `divide(x, y)`. - Start debugging. Notice `x = 10`, `y = 0`. Step into the function. - On the line `result = a / b`, you will get a **ZeroDivisionError** (because y=0). The debugger will show an exception popup or you will see the error in the Debug Console. - You can then inspect that `a=10`, `b=0` and realize the cause. The call stack shows the path to this line. - This helps identify exactly where and why the exception occurs. - You can add a conditional breakpoint (for example, break in `divide` only if `b==0`) to catch these cases automatically【20†L303-L312】.

---

## 7. Virtual Environments in Practice: OS Commands and Best Practices

### 7.1 OS-specific Commands

Below is a summary of common venv commands on Windows, macOS, and Linux:

| Step | Windows (cmd.exe) | Windows (PowerShell) | macOS/Linux (bash/zsh) |
|---|---|---|---|
| **Create venv** | `py -m venv .venv` <br> `python -m venv .venv` | `python -m venv .venv` | `python3 -m venv .venv` |
| **Activate venv** | `.venv\Scripts\activate.bat` | `.\.venv\Scripts\Activate.ps1` | `source .venv/bin/activate` |

| Step | Windows (cmd.exe) | Windows (PowerShell) | macOS/Linux (bash/zsh) |
|---|---|---|---|
| **Deactivate venv** | `deactivate` | `deactivate` | `deactivate` |
| **Check Python** | `where python` | `where python` (or `Get-Command python`) | `which python3` (or `python --version`) |
| **Install package** | `pip install <package>` (in venv) | `pip install <package>` (in venv) | `pip install <package>` (in venv) |
| **Freeze deps** | `pip freeze > requirements.txt` | `pip freeze > requirements.txt` | `pip freeze > requirements.txt` |
| **Install from file** | `py -m pip install -r requirements.txt` | `python -m pip install -r requirements.txt` | `python3 -m pip install -r requirements.txt` |

*Sources:* These commands come from the official Python packaging guide【11†L209-L218】【11†L229-L238】 and Python docs【8†L98-L106】. Using `python3` on Unix ensures you run Python 3 (since some systems have `python` pointing to 2.x).

### 7.2 Workspace vs. Global Environments

- **Workspace (local) venv:** A venv folder inside your project. VS Code will automatically detect `.venv/` if it exists【6†L338-L347】. Prefer this for project isolation.
- **Global Python:** A system-wide Python interpreter. Might be used if you just run a quick script, but it mixes dependencies globally.

To view discovered Python environments, open the **Python** sidebar (on the left) and check **Environment Managers** or run **Python: Select Interpreter**【6†L323-L330】. You will see sections like *venv*, *Conda*, *Global*. You can also refresh the list via the refresh icon.

### 7.3 pip, requirements.txt, and pyproject.toml

- **requirements.txt:** A text file listing your dependencies. Format is one package per line (e.g. `requests==2.25.1`). The pip docs describe it as *"a list of items to be installed by pip"*【28†L90-L99】. Use `pip install -r requirements.txt` to reinstall them.
- **pip freeze:** Generates a list of installed packages and their versions, which you can redirect to `requirements.txt` (e.g. `pip freeze > requirements.txt`). This is useful for pinning exact versions.
- **pyproject.toml:** A newer standard (PEP 518) for Python project metadata and build requirements. For simple projects, you can omit it and just use `requirements.txt`. If your project is a library or uses a build system (like Poetry or Flit), then `pyproject.toml` would define dependencies and tools. For this lesson, assume a `requirements.txt` workflow.

### 7.4 Best Practices

- **Isolate per project:** *Always* create and activate a venv for each project【8†L51-L59】.
- **No global installs:** Do not `pip install` project packages globally on your machine. That can break other projects or system tools.
- **.gitignore:** Add your venv folder to `.gitignore`. For example:

```
# Python venv
.venv/
```

This ensures you don't commit the entire environment to source control.

- **Reproducibility:** Share only `requirements.txt` (or `pyproject.toml`) and code. Colleagues/CI can recreate the environment by running the same venv creation and `pip install -r` commands.
- **Update pip:** Occasionally run `python -m pip install --upgrade pip setuptools` inside the venv to get security/bug fixes.
- **Interpreter consistency:** If you upgrade Python (e.g. from 3.9 to 3.10), recreate the venv or use `python -m venv --upgrade`. The venv's `python` should match your desired version【8†L98-L106】.
- **Debug in correct interpreter:** Ensure VS Code is using the same Python interpreter as your venv. If you change the venv, re-select it in VS Code.

---

## 8. Launch Configurations and Remote Debugging

### 8.1 Understanding `launch.json`

The `launch.json` file in `.vscode` contains debugging configurations. Key fields include:

- `"name"`: Label for this config (shown in the dropdown).
- `"type": "debugpy"`: Indicates the Python debugger.
- `"request": "launch"` (or `"attach"` for attaching to a running process).
- `"program": "${file}"`: The Python file to execute (use `"${file}"` for the current open file).
- `"console": "integratedTerminal"` or `"externalTerminal"`: Where program I/O goes【22†L617-L624】.
- `"args": []`: Command-line arguments to pass.
- `"cwd":` (current working directory), `"env":` for environment variables, etc.

For example, a `launch.json` to debug a script with arguments might look like:

```
{
  "name": "Debug with Args",
  "type": "debugpy",
  "request": "launch",
```

```
  "program": "${file}",
  "console": "integratedTerminal",
  "args": ["--mode", "test"]
}
```

You can have multiple configurations in `launch.json` (e.g. separate ones for different scripts). Choose which to run via the debug panel drop-down or the status bar item (the debug name appears after you start).

### 8.2 Remote Debugging Basics (Optional)

VS Code also supports debugging Python running on a **remote machine**. The steps involve:

1. **Install debugpy on remote:** `pip install debugpy`.
2. **On remote machine:** Modify your Python code to listen for a debugger, or launch via `debugpy`. For example:

```
import debugpy
debugpy.listen(("0.0.0.0", 5678))
print("Waiting for debugger attach")
debugpy.wait_for_client()
# Now do normal work...
```

Or run your script with debugpy:

```
python3 -m debugpy --listen 5678 --wait-for-client myscript.py
```

1. **SSH Tunnel (security):** If needed, set up an SSH tunnel so VS Code on your local machine can connect to the remote port securely【22†L473-L481】【22†L501-L509】. For example:

```
ssh -L 5678:localhost:5678 user@remote_ip
```

1. **VS Code attach config:** In your local `launch.json`, add an **Attach** configuration:

```
{
  "name": "Python: Attach (Remote)",
  "type": "debugpy",
  "request": "attach",
  "host": "localhost",
  "port": 5678,
  "pathMappings": [
    { "localRoot": "${workspaceFolder}", "remoteRoot": "." }
```

```
    ]
}
```

1. **Run debugger:** Start the remote Python program (it will wait at `wait_for_client()`), then select "Attach (Remote)" in VS Code and press F5. VS Code will connect to the remote debugpy server and hit any breakpoints you set locally【22†L593-L601】.

   The remote debugging documentation emphasizes security (use SSH, secure hosts) and ensuring source code matches on both machines【22†L473-L476】【22†L584-L593】.

### 8.3 Debugging Tests

VS Code's Python extension has built-in support for testing frameworks (e.g. unittest, pytest). You can run and debug tests by configuring the test explorer. If your test fails or hangs, you can set breakpoints in the test code and click the debug icon next to the test in the Testing view. This will launch a debug session for that test.

For detailed test debugging scenarios, refer to the Python testing documentation. In summary, **debugging a test** is similar to debugging any script: VS Code runs the test under debugpy, and you step through it.

---

## 9. Troubleshooting & Best Practices

1. **No Interpreter Found:** If VS Code shows "No interpreter" or "Python not found", ensure your Python is installed and on PATH. Restart VS Code after installing Python. Use **Python: Select Interpreter** to choose the correct one【6†L323-L330】.
2. **Environment Not Detected:** If your venv isn't listed, try refreshing the Environment Managers view or reloading VS Code. Check that the venv folder contains `pyvenv.cfg` and a `python` executable – otherwise recreate it【6†L352-L360】【8†L99-L107】.
3. **Breakpoints Not Hit:** Common causes:
4. You set the breakpoint on a line that never executes (e.g. inside an `if False:`).
5. You changed code but are still running old code (save the file, restart debugging).
6. You are debugging the wrong interpreter: e.g. VS Code is using a different Python than expected.
7. **Debug Console Issues:** The Debug Console cannot accept standard input from `input()` calls. Use an Integrated/External Terminal for programs that prompt the user【22†L617-L624】.
8. **Exception On Breakpoint:** A breakpoint that can't be registered (e.g. in optimized code) turns gray【15†L270-L278】. This may happen if you edit code while it's running in a session without live editing. Stop and restart the session.
9. **Extension Conflicts:** If multiple Python extensions are installed, disable extras. Only the Microsoft Python extension (and its debugpy) is needed.
10. **Slow Debugging:** If stepping is too fast or difficult to watch, use **Pause** (F5 while running) and then step.

11. **Detached Processes:** If you start a subprocess (e.g. with `multiprocessing`), by default the debugger may not automatically attach. You might need to configure *subProcess* launch option or attach to the new process.

**Troubleshooting Checklist**

- [ ] **Interpreter & Env:** Verify the correct Python interpreter (venv) is selected (status bar) and matches your project's venv.
- [ ] **Breakpoint Placement:** Make sure breakpoints are on executable lines and the code is saved.
- [ ] **Configuration:** Check `launch.json` settings (program path, working directory, console type).
- [ ] **Dependencies:** Ensure all required packages are installed in the venv (no ImportError at runtime).
- [ ] **Restart VS Code:** Sometimes after environment changes, a reload (`Ctrl+R` or command palette: Developer: Reload Window) fixes detection issues.
- [ ] **Check Output:** Look at the Debug Console for error messages. Stack traces will show where and why code failed.

---

## 10. Practice Exercises

1. **Set up a Virtual Environment (3 points)**

2. In your project folder, create a venv named `testenv` using the command for your OS. Activate it.

3. Install the `requests` package inside the venv.
4. Check that `requests` is installed by opening a Python REPL (`python`) and trying `import requests`.

*Answer:*

- Windows CMD: `py -m venv testenv` & `testenv\Scripts\activate.bat` 【11†L209-L218】【11†L229-L238】.
- macOS/Linux: `python3 -m venv testenv` & `source testenv/bin/activate` 【11†L209-L218】【11†L229-L238】.

- Then `pip install requests`. Verify with `python -c "import requests; print(requests.__version__)"`.

- **Select Interpreter in VS Code (2 points)**

- After creating `testenv`, how do you tell VS Code to use it?

- *Answer:* Open a Python file, click the Python version in the bottom status bar, then choose the interpreter pointing to `testenv`. Or use Command Palette: `Python: Select Interpreter` 【6†L323-L330】.

- **Debug a Function (4 points)**
  Given the following code in `calc.py`:

```
def multiply(a, b):
    return a * b   # (A)
result = multiply(3, 5)   # (B)
print("Product is", result)
```

- Set a breakpoint at line (A) and start debugging. What are the values of `a` and `b` when the breakpoint is hit?
*Answer:* `a = 3`, `b = 5`. The VARIABLES pane will show these at line (A).

1. **Conditional Breakpoint (3 points)**
   Modify the above code so that the breakpoint at line (B) only triggers when `result > 10`.

2. *Answer:* Right-click the red breakpoint circle and choose **Edit Breakpoint**. Add the condition `result > 10`. Now it will only pause if that condition is true. Alternatively, use *Hit Count* or log condition.

3. **Debugging with Input (3 points)**
   For this code:

```
name = input("Name? ")
print("Hi,", name)
```

- Which launch.json “console” type should you use to allow typing the name?
*Answer:* Use `"console": "integratedTerminal"` (or externalTerminal). The Debug Console alone doesn’t accept `input()` 【22†L617-L624】.

1. **Dependencies (5 points)**

2. After installing packages, what command creates `requirements.txt`?

3. How do you install from it on Windows vs macOS/Linux?
   *Answer:* Use `pip freeze > requirements.txt`. Then on Windows: `py -m pip install -r requirements.txt`. On Unix/macOS: `python3 -m pip install -r requirements.txt` 【16†L15-L24】.

4. **Keyboard Shortcuts (2 points)**
   What are the default shortcuts for **Step Over** and **Step Into**?
   *Answer:* Step Over = **F10**; Step Into = **F11** 【15†L232-L240】.

5. **Launch Configuration (5 points)**
   Interpret this snippet from a `launch.json`:

```
{
  "name": "Attach Django",
  "type": "debugpy",
  "request": "attach",
  "port": 5678,
  "host": "localhost"
}
```

- What does this configuration do?
*Answer:* It attaches the VS Code debugger to a running Python process listening on port 5678 on localhost (e.g. a remote/long-running Django server started with debugpy)【22†L517-L524】.

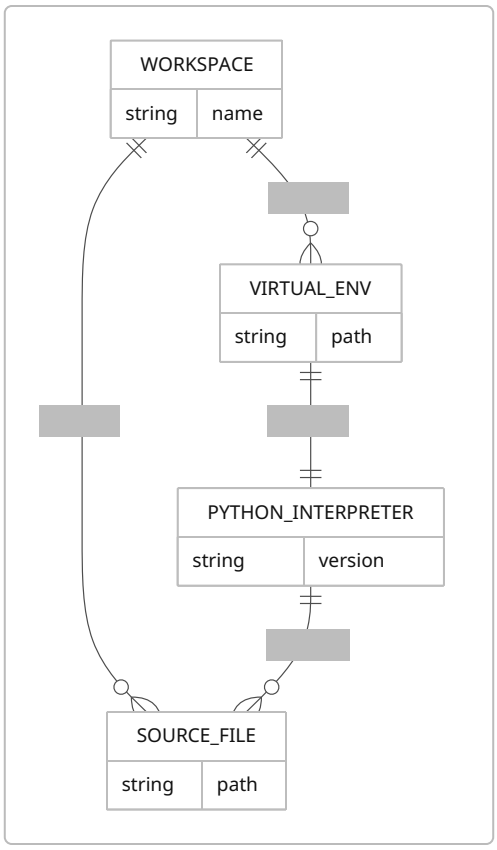
1. **Troubleshooting (4 points)**

2. If a breakpoint is gray and not stopping, what might be wrong?
   *Answer:* The breakpoint couldn't be registered, possibly because the code wasn't active or optimized. Solution: ensure the file is saved, restart debugging, or check if the code line is actually executed 【15†L270-L278】.

3. If VS Code won't attach to a remote debugpy session, what is one thing to check?
   *Answer:* Ensure you set up an SSH tunnel or correct port mapping, and that `pathMappings` in `launch.json` match the local and remote paths【22†L503-L513】.

---

```
flowchart LR
    A[Install VS Code] --> B[Install Python Extension]
    B --> C[Open/Create Python Project Folder]
    C --> D[Create virtual environment (venv)]
    D --> E[Activate virtual environment]
    E --> F[Select Python Interpreter in VS Code]
    F --> G[Set breakpoints and open Run & Debug view]
    G --> H[Start debugging session (F5)]
    H --> I[Step through code, inspect variables]
    I --> J[Fix bugs and stop debugging]
```




## References


- VS Code Debugging docs【15†L180-L187】【15†L232-L240】【20†L408-L417】 (official Microsoft docs on breakpoints, stepping, variables)
- VS Code Python docs【1†L231-L239】【6†L323-L330】【22†L473-L476】【22†L617-L624】 (Python extension debugging, launch.json, environments)
- Python `venv` docs【8†L51-L59】【8†L98-L106】 (official Python 3 documentation on virtual environments)
- Python Packaging User Guide【11†L209-L218】【11†L229-L238】【16†L15-L24】 (on creating venv, activating, pip, requirements)
- pip documentation【28†L90-L99】 (requirements file format)
- GeeksforGeeks【30†L29-L33】 (definition of debugger)


---